MSE800 Professional Software Development

Assessment 1

SmartDrive Car Rental System

Maintenance and Support

Student Name: Fan Fan

Student ID: 270958057

Student ID: 270958057@yoobeestudent.ac.nz

1. Purpose

This document addresses **Task 3: Software Evolution (LO2)**. It explains how SmartDrive can be updated, maintained, versioned, and kept compatible with older data.

The project is small, but it can grow in stages. New features can usually be added in the right layer instead of rewriting the whole codebase.

2. Handling Updates and Bug Fixes

Area	Recommended Practice	Suitable Tool or Method
Bug recording	Log each defect with affected workflow, expected behaviour, and verification notes	GitHub Issues
Code change review	Make changes in a separate branch and review them before merging	GitHub branches and pull requests
Documentation updates	Update diagrams, README, and user document when behaviour changes	Repository documentation plus shared notes
Team-scale project tracking	Track assigned work, priorities, and progress across iterations	Jira, with Confluence for supporting documentation

Small updates should start by finding the affected layer. A pricing issue belongs in services.py. A data retrieval issue belongs in repositories.py. A menu text issue belongs in cli.py.

Each change should begin with a recorded issue. GitHub Issues is suitable for logging bugs, feature requests, and maintenance tasks.

3. Adding New Features

Future features can be added in stages. Payment history, overlap checks, and richer recommendation rules could all be added without replacing the current structure.

A practical workflow is to create a feature branch, make the change in the correct layer, test the affected flow, and then merge it back through a reviewed pull request.

4. Versioning Strategy

Version Type	Meaning in This Project	Example
Patch	Small fixes that do not change the user	v1.0.1

	workflow	
Minor	Visible new features or controlled behaviour improvements	v1.1.0
Major	Large redesign of structure, data flow, or interaction model	v2.0.0

A lightweight semantic versioning style would work well:

- Patch version changes for small fixes that do not change the user workflow.
- Minor version changes for new features such as payment support or improved recommendations.
- Major version changes for significant redesigns to structure, data storage, or interaction flow.

Each version can be marked with a GitHub tag and a short release note. A changelog should record what changed, what was fixed, and whether the database was affected.

5. Backward Compatibility

Compatibility Area	Strategy
Database schema	Keep startup checks and migration logic in bootstrap.py.
Existing bookings	Preserve existing booking statuses and add new statuses carefully.
CLI workflow	Avoid changing menu numbers unless documentation and tests are updated together.
Data fields	Prefer adding new fields over renaming or deleting existing fields immediately.

Backward compatibility already matters because older SQLite databases may not match the latest schema. The bootstrap logic helps by checking the database and adding missing user contact columns at startup.

Schema changes should be additive where possible. Adding a column is safer than renaming or deleting one immediately.

6. Long-Term Software Engineering Value

The system will be easier to maintain if responsibilities stay separated and names stay consistent. Clear workflows and documentation also make future changes less risky.

In practice, maintainability is not only about code. It also depends on version control, issue tracking, documented decisions, and testing after each meaningful change.